CS/CE 224/272 - Object Oriented Programming and Design Methodology

Nadia Nasir

Habib University, Fall 2025

OOP Concepts

There are four main OOP concepts. These are:

Abstraction
Encapsulation
Inheritance
Polymorphism

These are also called as four pillars of Object Oriented Programming.

Abstraction

Abstraction focuses on simplifying complex systems by showing only the essential features and hiding the intricate implementation details. It allows you to represent the "what" an object does rather than the "how" it does it.

- **Key aspects:**

  Hiding complexity: Users interact with a simplified interface without needing to understand the underlying mechanisms.

  Focus on essential details: Only relevant information is exposed to the outside world.

  Achieved through: Abstract classes and interfaces, which define a contract for behavior without providing a full implementation.

Example: Imagine you're designing software for different types of vehicles. You might start with an interface:

```
// Abstraction: Interface defines the higher-level concept
interface Vehicle {
void startEngine();
void stopEngine();
}
```

The Vehicle interface specifies that any vehicle should be able to start and stop its engine. It doesn't care how these actions are performed – that's up to the specific types of vehicles to implement.

Encapsulation

A class describes a set of objects with the same behavior. For example, a Car class describes all passenger vehicles that have a certain capacity and shape.

A class describes a set of objects with the same behavior. For example You can drive a car by operating the steering wheel and pedals, without knowing how the engine works. Similarly, you use an object through its member functions. The implementation is hidden.

All you need to know is the public interface: the specifications for the member functions that you can invoke. **The process of providing a public interface, while hiding the implementation details, is called encapsulation.**

You will want to use encapsulation for your own classes. When you define a class, you will specify the behavior of the public member functions, but you will hide the implementation details.

Encapsulation is also a benefit for the implementor of a class. When working on a program that is being developed over a long period of time, it is common for implementation details to change, usually to make objects more efficient or more capable. Encapsulation is crucial to enabling these changes.

When the implementation is hidden, the implementor is free to make improvements. Because the implementation is hidden, these improvements do not affect the programmers who use the objects.

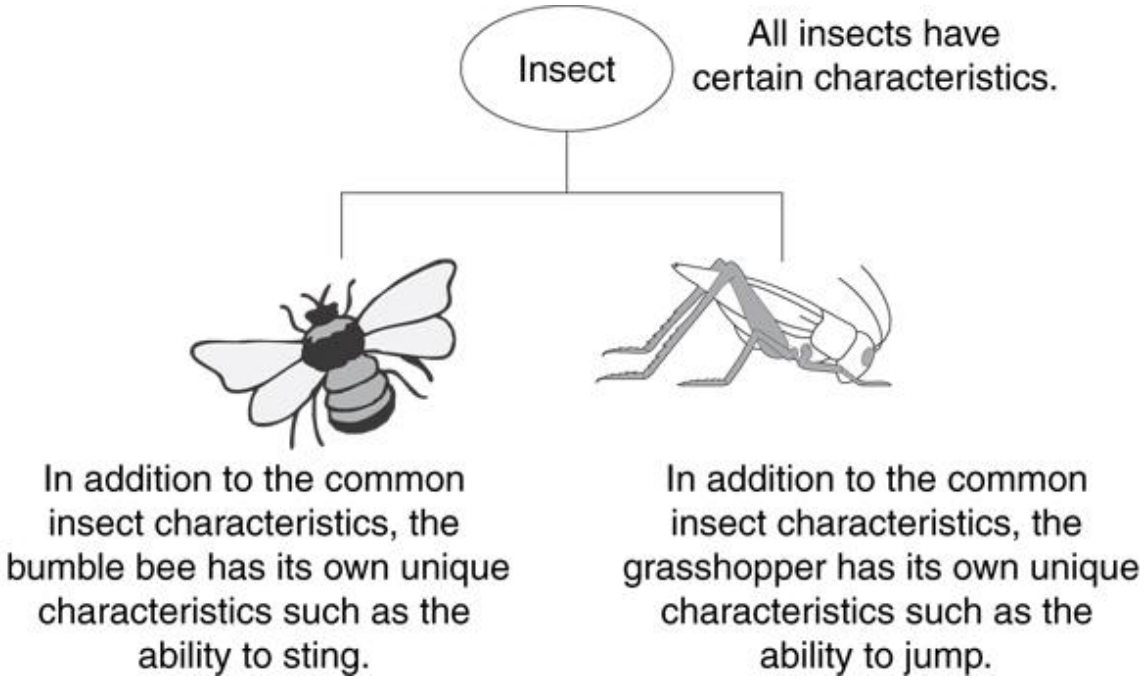
Inheritance

C++

What Is Inheritance?

Provides a way to create a new class from an existing class
The new class is a specialized version of the existing class

Example: Insect Taxonomy



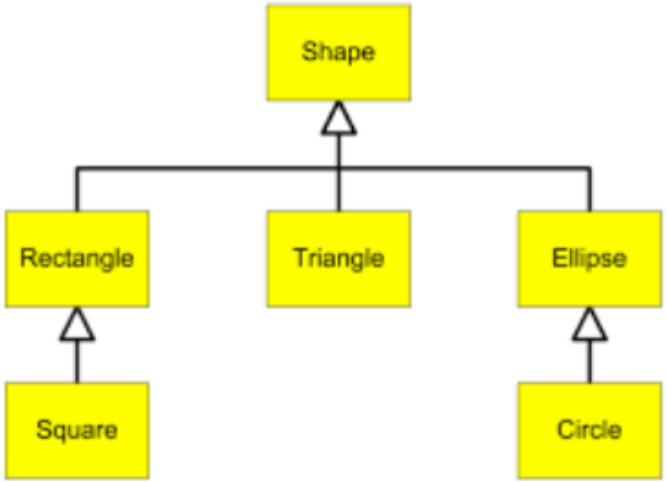
The "is a" Relationship

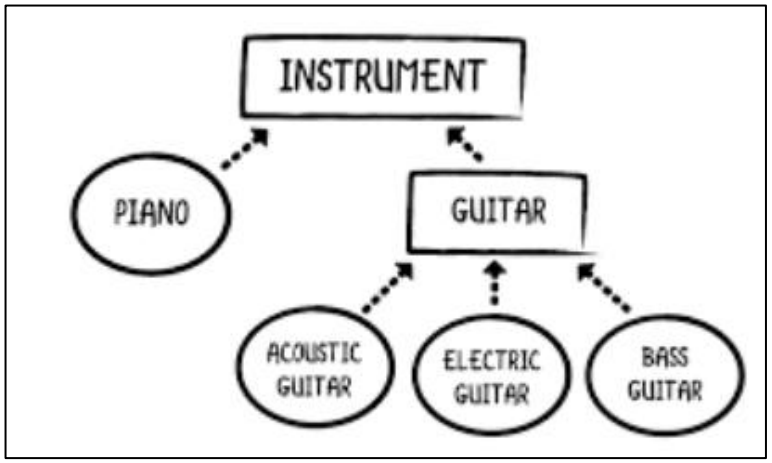
- Inheritance establishes an "is a" relationship between classes.
  - A poodle is a dog
  - A car is a vehicle
  - A flower is a plant
  - A football player is an athlete

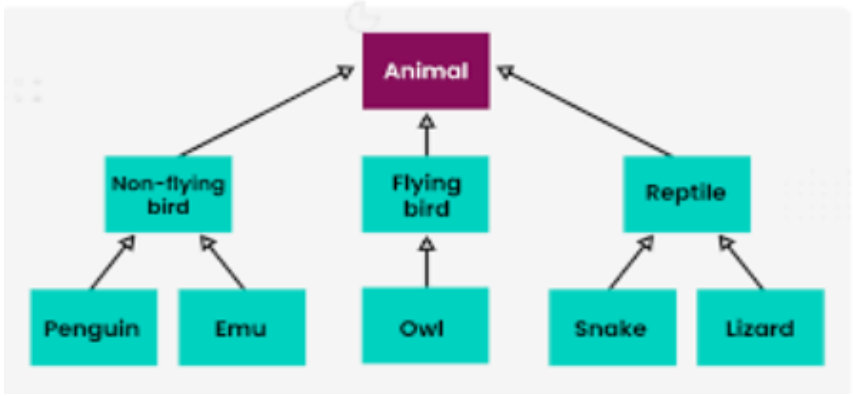
The class whose properties are inherited by other class is called the **Parent** or **Base** or **Super** class.

The class which inherits properties of other class is called **Child** or **Derived** or **Sub** class

Examples


Shape
Rectangle
Triangle
Ellipse
Square
Circle


INSTRUMENT
PIANO
GUITAR
ACOUSTIC GUITAR
ELECTRIC GUITAR
BASS GUITAR

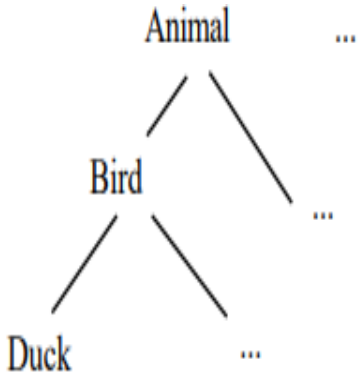

Animal
Non-flying bird
Flying bird
Reptile
Penguin
Emu
Owl
Snake
Lizard

Inheritance Vocabulary

- OOP Hierarchy
- Superclass / Subclass
- Inheritance
- Overriding
- "is a" –
  - an instance of a subclass is a instance of the superclass. The subclass is just a refined form of the superclass.
  - A subclass instance has all the properties that instances of the superclass are supposed to have, and it may have some additional properties as well.

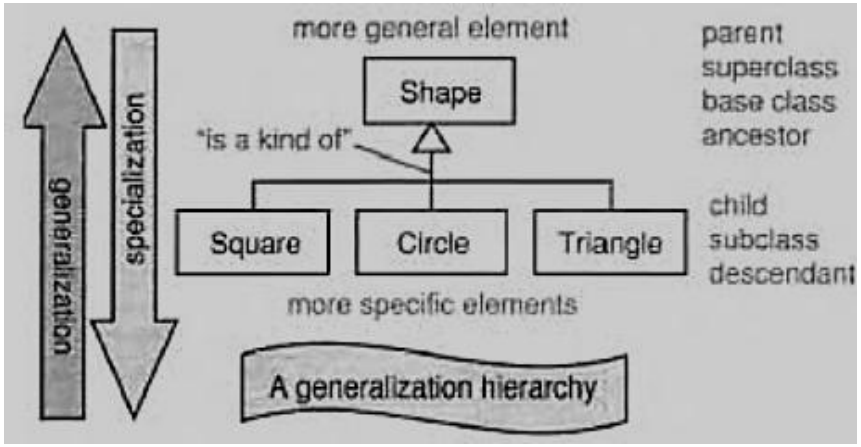
Hierarchy of classes

- By doing this, we get a hierarchy of classes.
- Classes in OOP are arranged in a tree-like hierarchy.
- "superclass" is the class above it in the tree.
- The classes below a class are its "subclasses."
- A hierarchy is useful if there are several classes which are fundamentally similar to each other.



General vs. Specific

The semantics of the hierarchy are that classes have all the properties of their superclasses. In this way the hierarchy is general up towards the root and specific down towards its leaves.



Superclass has fewer properties, is less constrained, is more general (confusingly, the word "super" can suggests a class with more properties)

Subclass has more properties, is more constrained, is more specific

Is-a vs. Has-a

- The subclass is constrained to be an "is a" specialization of the superclass.
- The subclass is merely a version of the superclass category.
- This relationship is so constraining... that's one reason why subclassing opportunities are rare.
  - e.g. Boat isa Vehicle, Chicken isa Bird isa Animal
- Contrast to the much more common "has-a" relationship where one object has a pointer to another
  - e.g. Boat has-a passenger, Game has-a Tetris piece, University has-a Student has-a Advisor

Purpose of inheritance

- Code Reusability
- Method Overriding (Hence, Runtime Polymorphism.)
- Use of Virtual Keyword

Basic Syntax of Inheritance

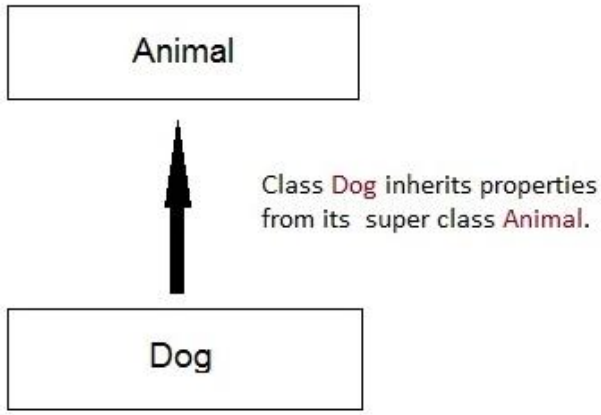
class Subclass_name : access_mode Superclass_name

```
class Animal
{
  ... .. ...
};

class Lion : public Animal
{
  ... .. ...
};

class Bear : public Animal
{
  .... .. ...
};

class Rabbit : public Animal
{
  ... .. ...
};
```



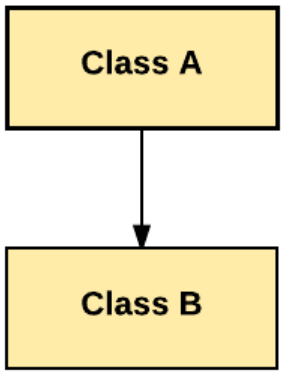
Inheritance Access Specifiers

	Derived Class	Derived Class	Derived Class
Base class	**Public Mode**	**Private Mode**	**Protected Mode**
Private	Not Inherited	Not Inherited	Not Inherited
Protected	Protected	Private	Protected
Public	Public	Private	Protected

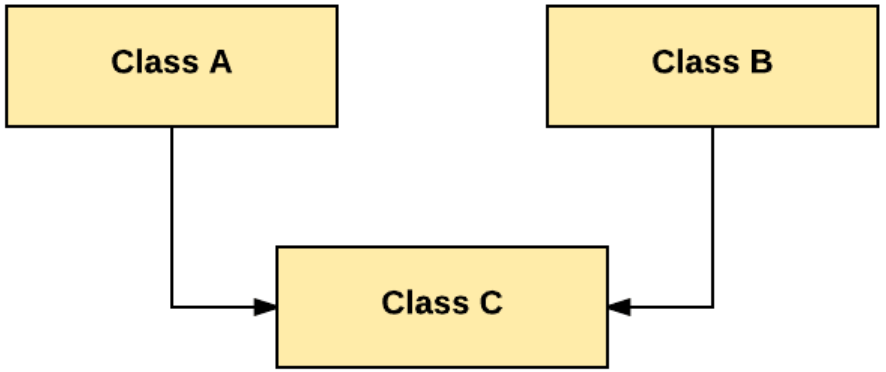
```
class base
{
            public:
                        int x;
            protected:
                        int y;
            private:
                        int z;
};
class publicDerived: public base
{
            // x is public
            // y is protected
            // z is not accessible from publicDerived
};
class protectedDerived: protected base
{
            // x is protected
            // y is protected
            // z is not accessible from
protectedDerived
};
class privateDerived: private base
{
            // x is private  //object level
            // y is private  // object level
            // z is not accessible from privateDerived
}
```

Types of Inheritance

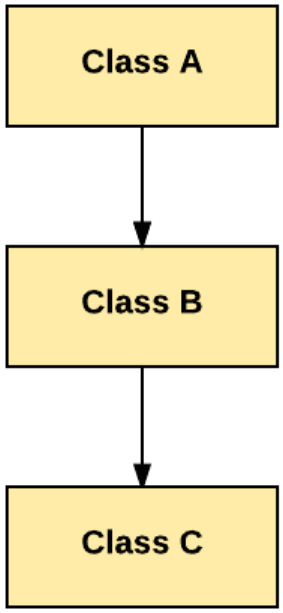
Single Inheritance



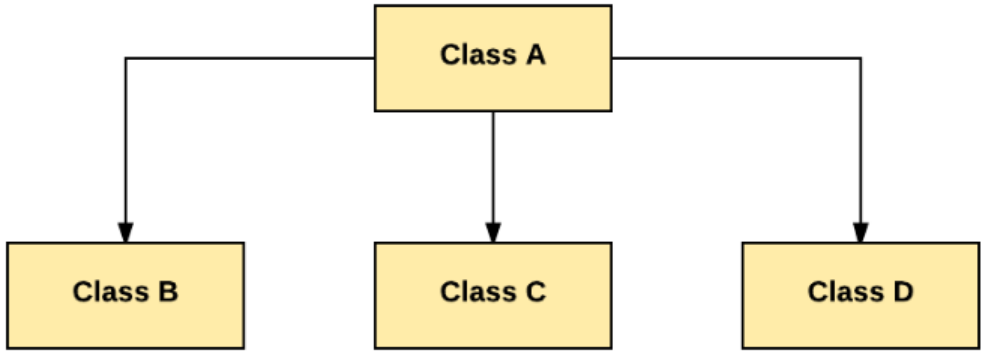
Multiple Inheritance



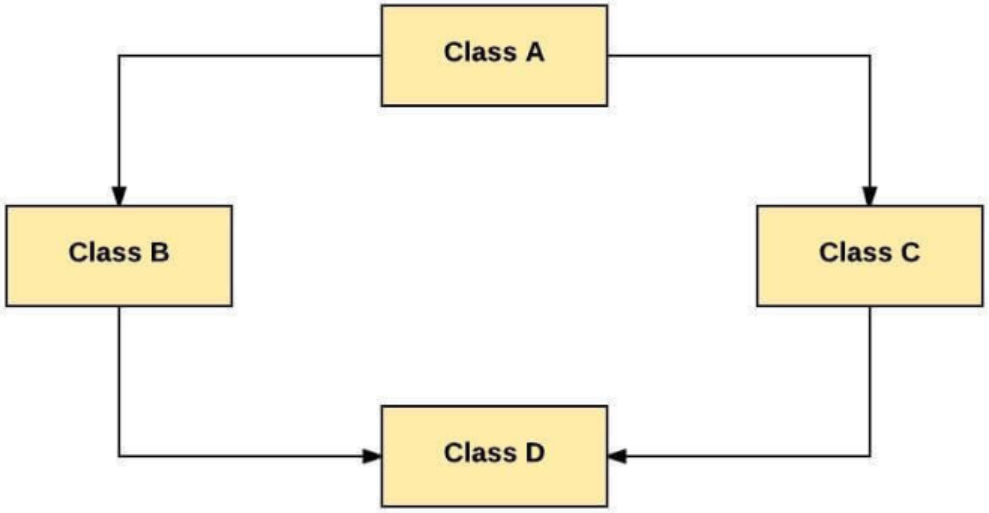
Multilevel Inheritance



Hierarchical Inheritance



Hybrid Inheritance



Order of Constructor Call with Inheritance in C++

Whether derived class's default constructor is called or parameterized is called, base class's default constructor is always called inside them.

To call base class's parameterized constructor inside derived class's parameterized constructor, we must mention it explicitly while declaring derived class's parameterized constructor.

Functions that are never Inherited

Constructors and Destructors are never inherited and hence never override.
Also, assignment operator = is never inherited. It can be overloaded but can't be inherited by sub class.